Exercise 5

Backtracking: Sum Of Subsets

Submitted by:
Immanuel Joyson P
128157020
Roll no:19

Objective:
To study the working principle of backtracking algorithms using sum of subset problem.

Algorithm:

Algorithm Initialize(A, x, s, r, count)

1. s = r = count = 0
2. for i = 1 to n
3. x[i] = 0
4. r = r + A[i]
5. Return

Algorithm SubsetSum(A, x,n, m, k, s, r, count)

1. if s == m
2. for i = 1 to k − 1
3. if (x[i] == 1)
4. print A[i]
5. end if
6. end for
7. return

8. end if
9. if s + A[k] ≤ m
10. x[k] = 1
11. count = count + 1
12. SubsetSum(A, x, n, m, k + 1, s + A[k], r − A[k])

13. if s + r − A[k] ≥ m
14. x[k] = 0
15. count = count + 1
16. SubsetSum(A, x, n, m, k + 1, s, r − A[k])
17. end if
18. end if
19. Return

Code:

```
#include<iostream>
using namespace std;


class arr{
        public:
                int a[10];
                int x[10];
                int count,n;

                arr(int num, int list[],int tar){
                        n=num;
                        count=0;
                        cout<<"input array : ";
                        for(int i=0;i<n;i++){
                        a[i]=list[i];
                        x[i]=0;
                        cout<<a[i]<<" ";
                        }
```

```
        cout<<endl<<"target : "<<tar<<endl;}

    void subsetSum(int m, int k ,int s,int r){
        if(s==m){
            cout<<"subarray is : ";
            for(int i=0;i<k;i++){
                if(x[i]==1)
                    cout<<a[i]<<" ";}
            cout<<endl<<"count : "<<count<<endl;
        return ;}
        if(s+a[k] <=m){
            x[k]=1;
            count++;

            subsetSum(m,k+1,s+a[k],r-a[k]);
            if(s+r-a[k]>=m){
                x[k]=0;
                count++;
                subsetSum(m,k+1,s,r-a[k]);}}}

    };


int main(){
    int l[]={5,8,10,12};
    arr obj(4,l,25);
    obj.subsetSum(25,0,0,35);          }
```

output:

```
input array : 5 8 10 12
target : 25
subarray is : 5 8 12
count : 5
```

```
input array : 5 8 10 12 13
target : 25
subarray is : 5 8 12
count : 5
subarray is : 12 13
count : 21
```

```
input array : 5 8 10 12 13 15
target : 30
subarray : 5 10 15
count : 14
subarray : 5 12 13
count : 17
subarray : 8 10 12
count : 24
```

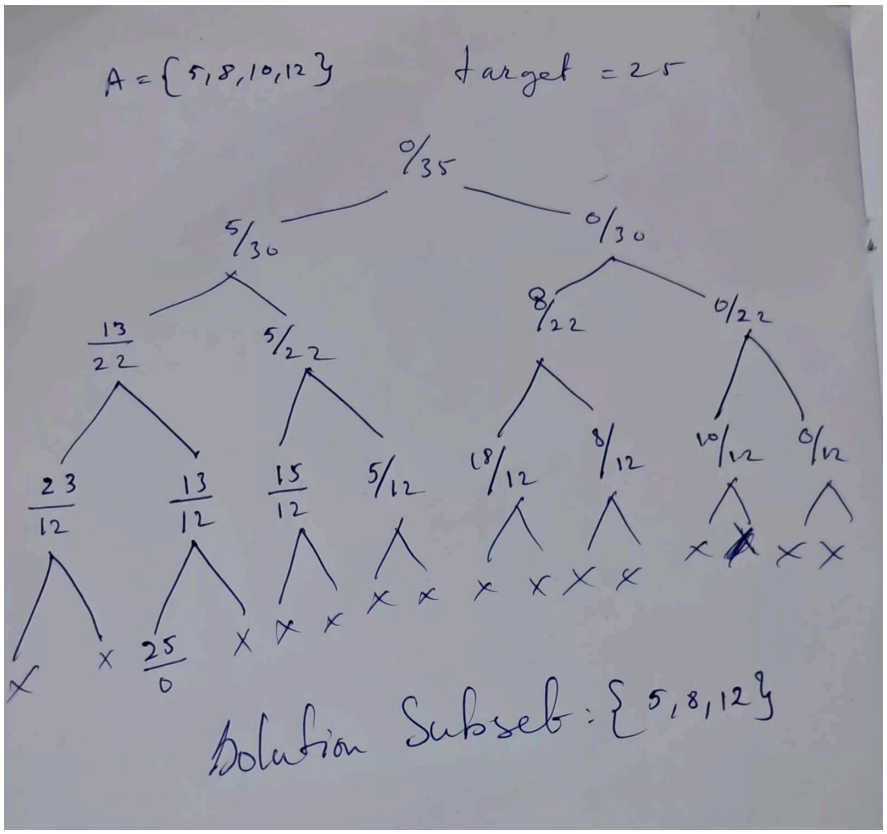
```
input array : 5 8 10 12 13 15 18
target : 30
subarray : 5 10 15
count : 17
subarray : 5 12 13
count : 21
subarray : 8 10 12
count : 30
subarray : 12 18
count : 51
```

```
input array : 5 8 10 12 13 15 18 22
target : 35
subarray : 5 8 10 12
count : 4
subarray : 5 8 22
count : 15
subarray : 5 12 18
count : 32
subarray : 8 12 15
count : 55
subarray : 10 12 13
count : 66
subarray : 13 22
count : 87
```

```
input array : 5 8 10 12 13 15 18 22 25
target : 35
subarray : 5 8 10 12
count : 4
subarray : 5 8 22
count : 15
subarray : 5 12 18
count : 33
subarray : 8 12 15
count : 58
subarray : 10 12 13
count : 71
subarray : 10 25
count : 82
subarray : 13 22
count : 99
```

```
input array : 5 8 10 12 13 15 18 22 25 28
target : 40
subarray : 5 8 12 15
count : 14
subarray : 5 10 12 13
count : 30
subarray : 5 10 25
count : 41
subarray : 5 13 22
count : 59
subarray : 8 10 22
count : 82
subarray : 10 12 18
count : 113
subarray : 12 13 15
count : 130
subarray : 12 28
count : 141
subarray : 15 25
count : 158
subarray : 18 22
count : 162
```

Hand Calculation:



Tabulation:

No of elements in set A	Set elements	Target sum	Solution subsets	No of solutions	No of nodes generated	Total nodes in space tree
4	{5,8,10,12}	25	{5,8,12}	1	10	32
5	{5,8,10,12,13}	25	{5,8,12}, {12,13}	2	18	64
6	{5,8,10,12,13, 15}	30	{5,10,15},{5,12,13}, {8,12,10}	3	28	128
7	{5,8,10,12,13, 15,18}	30	{5,8,10,12} , {5,8,22}, {5,12,18} , {8,12,15}, {10,12,13} , {10,25,}, {13,22}	4	40	256

8	{5,8,10,12,13, 15,18,22}	35	{5,8,10,12} , {5,8,22}, {5,12,18} , {8,12,15} , {10,12,13} , {13,22}	6	58	512
9	{5,8,10,12,13, 15,18,22,25}	35	{5,8,10,12} , {5,8,22}, {5,12,18} , {8,12,15} , {10,12,13} , {10,25} {13,22}	7	80	1024
10	{5,8,10,12,13, 15,18,22,25,28}	40	{5,8,12,15}, {5,10,12,13}, {5,10,25} , {5,13,22}, {8,10,22} , {10,12,18} , {12,13,15} , {12,28} , {15,25} , {18,22}	10	110	2048

Result:

The sum of subsets problem was implemented using the backtracking method. The program successfully generated all subsets whose sum equals the given target value for different input size . It was observed that as the number of elements increases , the total possible nodes in the state space tree grow rapidly. However due to pruning using bounding conditions , only the necessary nodes were generated . Hence backtracking efficiently reduces unnecessary computations while finding all valid solutions